Análisis y Desarrollo de Software (ADSO)

GC-F-004v.08

Servicio Nacional de Aprendizaje

TABLA DE **CONTENIDO**

01 FLUTTER - ARQUITECTURA

02 FLUTTER - DIO

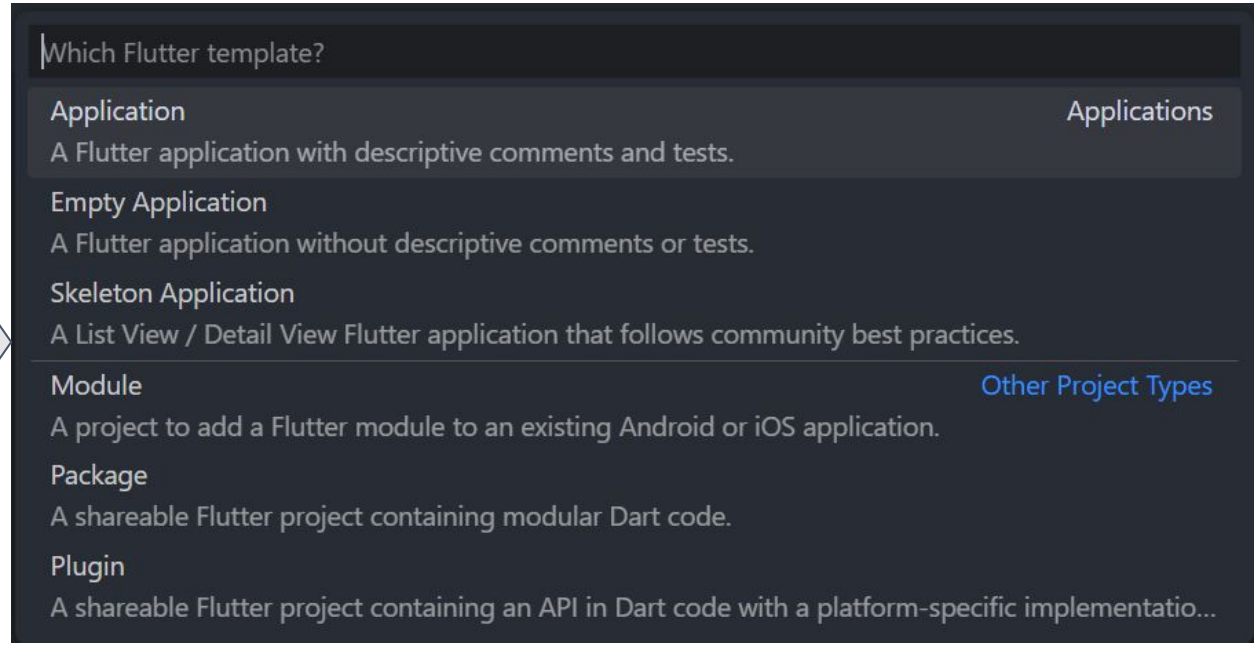
FLUTTER - APP ARCHITECTURE

Crear la siguiente estructura para el proyecto

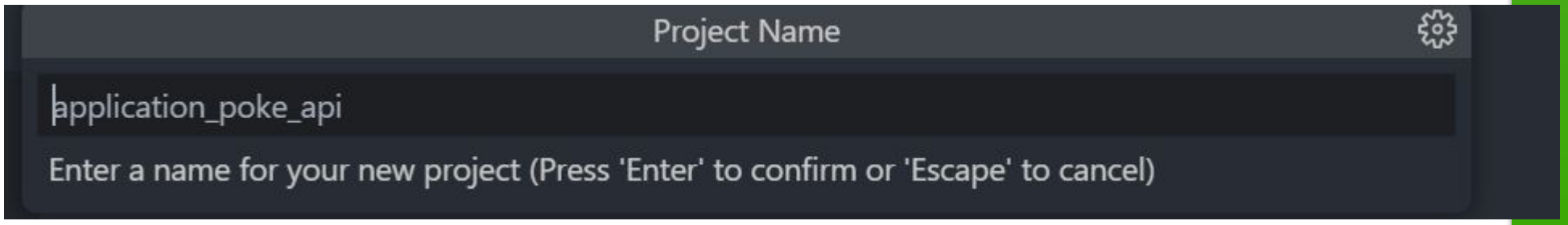
Servicio Nacional de Aprendizaje

PASO 1

PASO 2



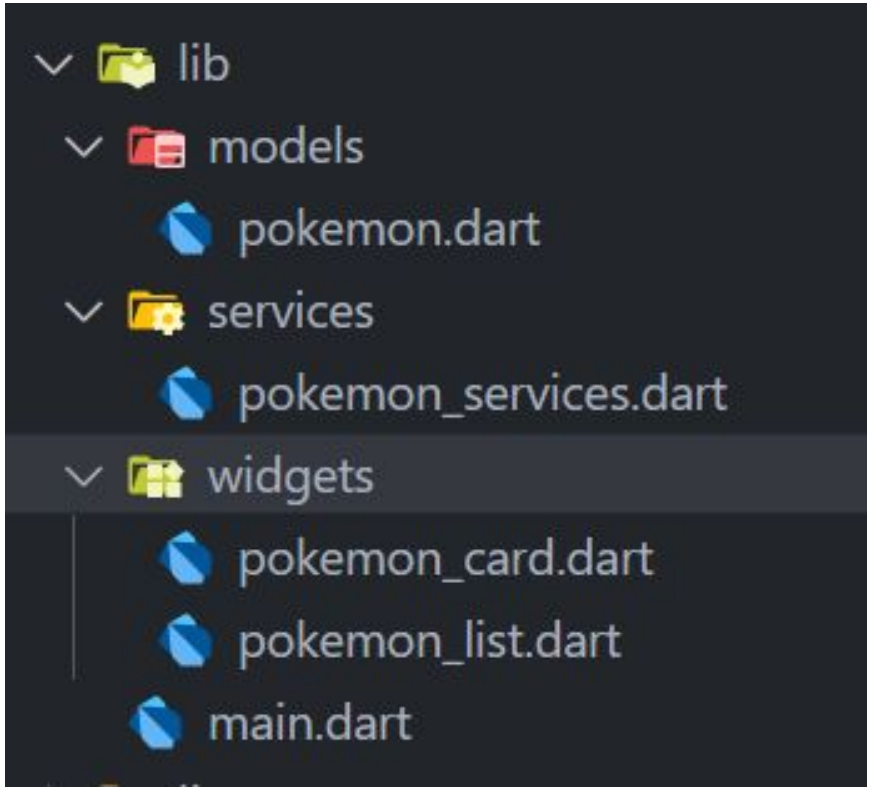
PASO 3



FLUTTER - 1. Estructura del proyecto

Crear la siguiente estructura para el proyecto

```
lib/
├── models/
│   └── pokemon.dart
├── services/
│   └── pokemon_service.dart
├── widgets/
│   ├── pokemon_card.dart
│   └── pokemon_list.dart
└── main.dart
```



FLUTTER - 2. Archivo pubspec.yaml

Validar el archivo pubspec.yaml, que tenga las dependencias activas.

```yaml
name: application_poke_api
description: "A new Flutter project app that consumes the Poke API."

publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  dio: ^5.4.0
  cached_network_image: ^3.3.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
```

FLUTTER - 3. models/pokemon.dart

Ajustar el archivo models/pokemon.dart, con la siguiente clase.

```dart
class Pokemon {
  final int id;
  final String name;
  final String imageUrl;
  final List<String> types;
  final int height;
  final int weight;

  Pokemon({
    required this.id,
    required this.name,
    required this.imageUrl,
    required this.types,
    required this.height,
    required this.weight,
  });
```

```dart
  factory Pokemon.fromJson(Map<String, dynamic> json) {
    return Pokemon(
      id: json['id'],
      name: json['name'],
      imageUrl: json['sprites']['front_default'] ?? '',
      types: (json['types'] as List)
          .map((type) => type['type']['name'] as String)
          .toList(),
      height: json['height'],
      weight: json['weight'],
    );
  }

  String get capitalizedName => name[0].toUpperCase() + name.substring(1);

  String get formattedId => '#${id.toString().padLeft(3, '0')}';
}
```

FLUTTER - 4.

Ajustar el archivo services/pokemon_service.dart, con la siguiente clase.

lib > services > pokemon_services.dart > PokemonService > fetchPokemons

```dart
import 'package:dio/dio.dart';
import '../models/pokemon.dart';

class PokemonService {
  final Dio _dio = Dio(BaseOptions(
    baseUrl: 'https://pokeapi.co/api/v2/',
    connectTimeout: const Duration(seconds: 10),
    receiveTimeout: const Duration(seconds: 10),
  )); // BaseOptions // Dio

  Future<List<Pokemon>> fetchPokemons({int limit = 20, int offset = 0}) async {
    try {
      final response = await _dio.get(
        'pokemon',
        queryParameters: {'limit': limit, 'offset': offset},
      );

      if (response.statusCode == 200) {
        final List<dynamic> results = response.data['results'];

        // Fetch details for each Pokémon
        final List<Future<Pokemon>> pokemonFutures =
            results.map((pokemon) async {
          final pokemonResponse = await _dio.get(pokemon['url']);
          return Pokemon.fromJson(pokemonResponse.data);
        }).toList();

        final List<Pokemon> pokemons = await Future.wait(pokemonFutures);
        return pokemons;
      } else {
        throw Exception('Failed to load pokemons');
      }
    } on DioException catch (e) {
      if (e.response != null) {
        throw Exception('Error: ${e.response!.statusCode}');
      } else {
        throw Exception('Network error: ${e.message}');
      }
    }
  }
```

```dart
  Future<Pokemon> fetchPokemonById(int id) async {
    try {
      final response = await _dio.get('pokemon/$id');

      if (response.statusCode == 200) {
        return Pokemon.fromJson(response.data);
      } else {
        throw Exception('Failed to load pokemon');
      }
    } on DioException catch (e) {
      if (e.response != null) {
        throw Exception('Error: ${e.response!.statusCode}');
      } else {
        throw Exception('Network error: ${e.message}');
      }
    }
  }
}
```

FLUTTER - 4.

services/pokemon_service.dart

Instalar las siguientes dependencias:

- **Abrir una consola, digitar**

```
\App-Flutter\application_poke_api> flutter pub add dio
```

FLUTTER - 5. widgets/pokemon_card.dart

Ajustar el archivo widgets/pokemon_card.dart, con la siguiente clase.

lib > widgets > pokemon_card.dart > PokemonCard > _getTypeColor

```dart
import 'package:flutter/material.dart';
import 'package:cached_network_image/cached_network_image.dart';
import '../models/pokemon.dart';

class PokemonCard extends StatelessWidget {
  final Pokemon pokemon;
  final VoidCallback onTap;

  const PokemonCard({
    super.key,
    required this.pokemon,
    required this.onTap,
  });

```

```dart
Color _getTypeColor(String type) {
  final colors = {
    'normal': Colors.brown[400],
    'fire': Colors.red[400],
    'water': Colors.blue[400],
    'electric': Colors.yellow[600],
    'grass': Colors.green[400],
    'ice': Colors.cyan[300],
    'fighting': Colors.orange[800],
    'poison': Colors.purple[400],
    'ground': Colors.amber[600],
    'flying': Colors.indigo[300],
    'psychic': Colors.pink[400],
    'bug': Colors.lightGreen[500],
    'rock': Colors.grey[600],
    'ghost': Colors.deepPurple[400],
    'dragon': Colors.indigo[800],
    'dark': Colors.brown[800],
    'steel': Colors.blueGrey[400],
    'fairy': Colors.pink[200],
  };

  return colors[type] ?? Colors.grey[400]!;
}
```

FLUTTER - 5. widgets/pokemon_card.dart

Ajustar el archivo widgets/pokemon_card.dart, con la siguiente clase.

```dart
@override
Widget build(BuildContext context) {
  return Card(
    elevation: 4,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    child: InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // Header with ID and Name
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Text(
                  pokemon.formattedId,
                  style: const TextStyle(
                    fontSize: 12,
                    fontWeight: FontWeight.bold,
                    color: Colors.grey,
                  ), // TextStyle
                ), // Text
                Text(
                  pokemon.capitalizedName,
                  style: const TextStyle(
                    fontSize: 18,
                    fontWeight: FontWeight.bold,
                  ), // TextStyle
                ), // Text
              ],
            ), // Row
```

```dart
            const SizedBox(height: 16),

            // Pokémon Image
            Center(
              child: CachedNetworkImage(
                imageUrl: pokemon.imageUrl,
                height: 100,
                width: 100,
                placeholder: (context, url) =>
                    const CircularProgressIndicator(),
                errorWidget: (context, url, error) => const Icon(Icons.error),
              ), // CachedNetworkImage
            ), // Center

            const SizedBox(height: 16),

            // Types
            Wrap(
              spacing: 8,
              children: pokemon.types.map((type) {
                return Chip(
                  label: Text(
                    type.toUpperCase(),
                    style: const TextStyle(
                      fontSize: 10,
                      color: Colors.white,
                      fontWeight: FontWeight.bold,
                    ), // TextStyle
                  ), // Text
                  backgroundColor: _getTypeColor(type),
                ); // Chip
              }).toList(),
            ), // Wrap
```

FLUTTER - 5. widgets/pokemon_card.dart

Servicio Nacional de Aprendizaje

Ajustar el archivo widgets/pokemon_card.dart, con la siguiente clase.

```dart
            const SizedBox(height: 8),

            // Stats
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceAround,
              children: [
                _buildStat('Height', '${pokemon.height / 10}m'),
                _buildStat('Weight', '${pokemon.weight / 10}kg'),
              ],
            ), // Row
          ],
        ), // Column
      ), // Padding
    ), // InkWell
  ); // Card
}
```

```dart

Widget _buildStat(String label, String value) {
  return Column(
    children: [
      Text(
        label,
        style: const TextStyle(
          fontSize: 12,
          color: Colors.grey,
        ), // TextStyle
      ), // Text
      Text(
        value,
        style: const TextStyle(
          fontSize: 14,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text
    ],
  ); // Column
}
}
```

FLUTTER - 5. widgets/pokemon_card.dart

Instalar las siguientes dependencias:

- **Abrir una consola, digitar**

```
\App-Flutter\application_poke_api> flutter pub add cached_network_image
```

FLUTTER - 6. widgets/pokemon_list.dart

Servicio Nacional de Aprendizaje

Ajustar el archivo 6. widgets/pokemon_list.dart , con la siguiente clase.

lib > widgets > pokemon_list.dart > PokemonList > PokemonList

```dart
import 'package:flutter/material.dart';
import '../models/pokemon.dart';
import 'pokemon_card.dart';

class PokemonList extends StatelessWidget {
  final List<Pokemon> pokemons;
  final Function(Pokemon) onPokemonTap;
  final bool isLoading;
  final VoidCallback? onLoadMore;

  const PokemonList({
    super.key,
    required this.pokemons,
    required this.onPokemonTap,
    this.isLoading = false,
    this.onLoadMore,
  });

```

```dart
  @override
  Widget build(BuildContext context) {
    return NotificationListener<ScrollNotification>(
      onNotification: (ScrollNotification scrollInfo) {
        if (onLoadMore != null &&
            !isLoading &&
            scrollInfo.metrics.pixels == scrollInfo.metrics.maxScrollExtent) {
          onLoadMore!();
        }
        return false;
      },
      child: Column(
        children: [
          Expanded(
            child: GridView.builder(
              gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
                crossAxisCount: 2,
                crossAxisSpacing: 8,
                mainAxisSpacing: 8,
                childAspectRatio: 0.8,
              ), // SliverGridDelegateWithFixedCrossAxisCount
              padding: const EdgeInsets.all(8),
              itemCount: pokemons.length + (isLoading ? 1 : 0),
              itemBuilder: (context, index) {
                if (index == pokemons.length) {
                  return const Center(child: CircularProgressIndicator());
                }

                final pokemon = pokemons[index];
                return PokemonCard(
                  pokemon: pokemon,
                  onTap: () => onPokemonTap(pokemon),
                ); // PokemonCard
              },
            ), // GridView.builder
          ), // Expanded
        ],
      ), // Column
    ); // NotificationListener
  }
}
```

FLUTTER - 7. main.dart

Ajustar el archivo 7. main.dart

lib > main.dart > _PokemonHomePageState > _loadPokemons

```dart
import 'package:flutter/material.dart';
import 'services/pokemon_services.dart';
import 'models/pokemon.dart';
import 'widgets/pokemon_list.dart';

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}
```

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'PokeAPI App',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ), // ThemeData
      home: const PokemonHomePage(),
    ); // MaterialApp
  }
}
```

FLUTTER - 7. main.dart

Ajustar el archivo 7. main.dart

```dart
class PokemonHomePage extends StatefulWidget {
  const PokemonHomePage({super.key});

  @override
  State<PokemonHomePage> createState() => _PokemonHomePageState();
}
```

```dart
class _PokemonHomePageState extends State<PokemonHomePage> {
  final PokemonService _pokemonService = PokemonService();
  final List<Pokemon> _pokemons = [];
  bool _isLoading = false;
  int _offset = 0;
  final int _limit = 20;

  @override
  void initState() {
    super.initState();
    _loadPokemons();
  }

  Future<void> _loadPokemons() async {
    if (_isLoading) return;

    setState(() {
      _isLoading = true;
    });
```

```dart

    try {
      final newPokemons = await _pokemonService.fetchPokemons(
        limit: _limit,
        offset: _offset,
      );

      setState(() {
        _pokemons.addAll(newPokemons);
        _offset += _limit;
        _isLoading = false;
      });
    } catch (e) {
      setState(() {
        _isLoading = false;
      });
      _showErrorSnackbar(e.toString());
    }
  }

```

FLUTTER - 7. main.dart

Ajustar el archivo 7. main.dart

```dart
void _showErrorSnackbar(String message) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text('Error: $message'),
      backgroundColor: Colors.red,
    ), // SnackBar
  );
}
```

```dart
void _onPokemonTap(Pokemon pokemon) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: Text(pokemon.capitalizedName),
      content: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Image.network(pokemon.imageUrl),
          const SizedBox(height: 16),
          Text('ID: ${pokemon.formattedId}'),
          Text('Height: ${pokemon.height / 10}m'),
          Text('Weight: ${pokemon.weight / 10}kg'),
          Text('Types: ${pokemon.types.join(', ')}'),
        ],
      ), // Column
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Close'),
        ), // TextButton
      ],
    ), // AlertDialog
  );
}
```

FLUTTER - 7. main.dart

Ajustar el archivo 7. main.dart

```
106
107   @override
108   Widget build(BuildContext context) {
109     return Scaffold(
110       appBar: AppBar(
111         title: const Text('PokéAPI App'),
112         backgroundColor: Theme.of(context).colorScheme.primary,
113         foregroundColor: Colors.white,
114       ), // AppBar
115       body: _pokemons.isEmpty && _isLoading
116           ? const Center(child: CircularProgressIndicator())
117           : PokemonList(
118               pokemons: _pokemons,
119               onPokemonTap: _onPokemonTap,
120               isLoading: _isLoading,
121               onLoadMore: _loadPokemons,
122             ), // PokemonList
123       floatingActionButton: FloatingActionButton(
124         onPressed: _loadPokemons,
125         child: const Icon(Icons.refresh),
126       ), // FloatingActionButton
127     ); // Scaffold
128   }
129 }
```

Servicio Nacional de Aprendizaje

FLUTTER - 8. Ejecutar

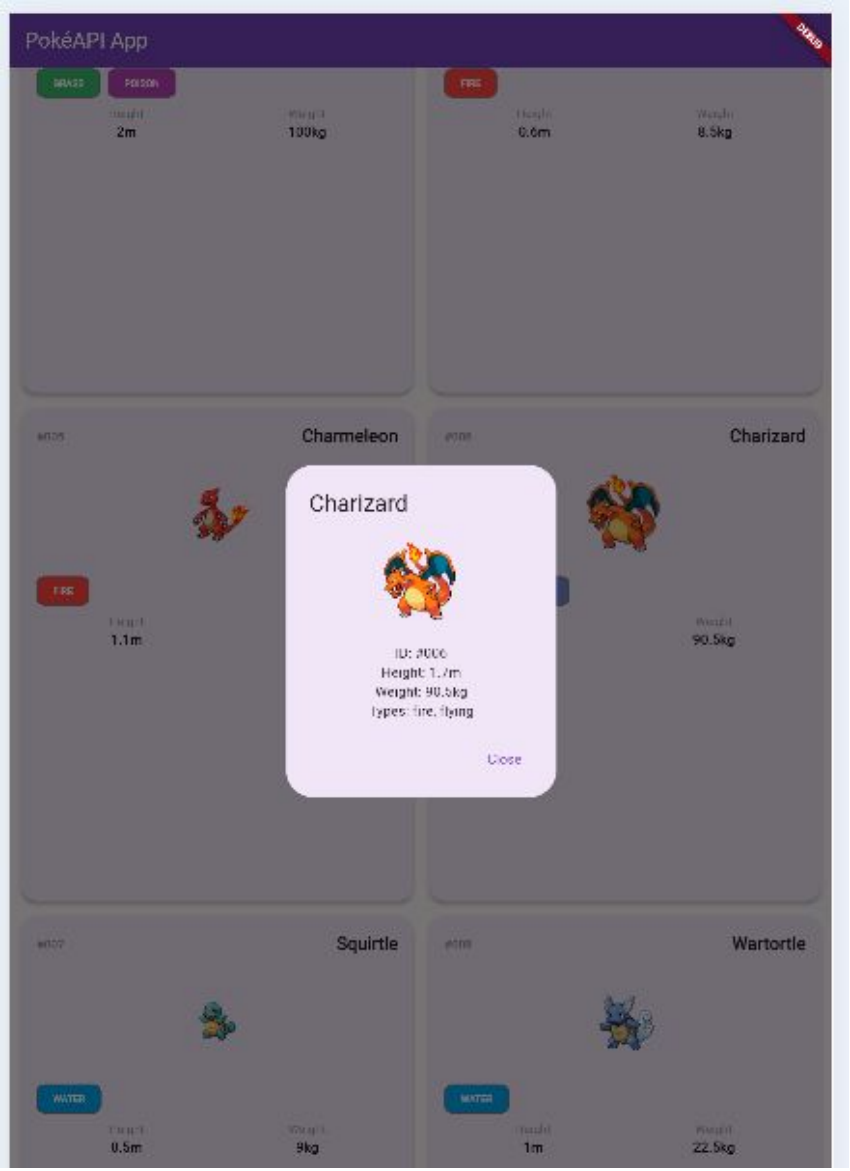
Ejecutar el proyecto

Para ejecutar la aplicación, asegúrate de ejecutar **flutter pub get** después de agregar las dependencias al pubspec.yaml

```
PS C:\Projects\App-Flutter\application_poke_api> flutter run
Connected devices:
Chrome (web) • chrome • web-javascript • Google Chrome 139.0.7258.155
Edge (web)   • edge   • web-javascript • Microsoft Edge 139.0.3405.111
[1]: Chrome (chrome)
[2]: Edge (edge)
Please choose one (or "q" to quit): 1
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...            39,5s
This app is linked to the debug service: ws://127.0.0.1:63949/YjJXbvHIakI=/ws
Debug service listening on ws://127.0.0.1:63949/YjJXbvHIakI=/ws
```

FLUTTER - 8. Ejecutar

Resultado



Servicio Nacional de Aprendizaje

02 FLUTTER - DIO

FLUTTER - DIO

¿Qué es DIO?

DIO es un poderoso cliente HTTP para Dart/Flutter que ofrece funcionalidades avanzadas más allá de las proporcionadas por el paquete http nativo de Dart. Es una de las librerías más populares para hacer peticiones HTTP en el ecosistema Flutter.

FLUTTER - DIO
Características principales de DIO

1. Interceptores

Permiten modificar las requests y responses globalmente:

```
dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) {
    // Modificar request antes de enviar
    options.headers['Authorization'] = 'Bearer token';
    return handler.next(options);
  },
  onResponse: (response, handler) {
    // Procesar response
    return handler.next(response);
  },
  onError: (DioException e, handler) {
    // Manejar errores
    return handler.next(e);
  },
));
```

FLUTTER - DIO
Características principales de DIO

2. Cancelación de requests

```
final cancelToken = CancelToken();
dio.get('/url', cancelToken: cancelToken);

// Cancelar la request
cancelToken.cancel('Request cancelled by user');
```

FLUTTER - DIO
Características principales de DIO

3. Timeout configurable

```
Dio(BaseOptions(
  connectTimeout: Duration(seconds: 5),
  receiveTimeout: Duration(seconds: 3),
  sendTimeout: Duration(seconds: 3),
));
```

FLUTTER - DIO
Características principales de DIO

4. Descarga y upload de archivos con progreso

```
// Download con progreso
dio.download(
  'https://example.com/file.zip',
  './file.zip',
  onReceiveProgress: (received, total) {
    print('${(received / total * 100).toStringAsFixed(0)}%');
  },
);
```

FLUTTER – DIO
Características principales de DIO

5. Transformers

Permiten transformar datos antes de enviar o después de recibir:

```
dio.transformer = BackgroundTransformer();
```

FLUTTER – DIO
Características principales de DIO

6. Soporte para FormData

```
FormData formData = FormData.fromMap({
  'name': 'John',
  'age': 30,
  'file': await MultipartFile.fromFile('./avatar.jpg'),
});

dio.post('/info', data: formData);
```

FLUTTER - DIO
Ejemplos de uso común

Configuración básica

```dart
import 'package:dio/dio.dart';

final dio = Dio(BaseOptions(
  baseUrl: 'https://api.example.com',
  connectTimeout: Duration(seconds: 5),
  receiveTimeout: Duration(seconds: 3),
  headers: {
    'Content-Type': 'application/json',
  },
));
```

FLUTTER - DIO
Ejemplos de uso común

Métodos HTTP

```
// GET
Response response = await dio.get('/users');

// POST
Response response = await dio.post(
  '/users',
  data: {'name': 'John', 'email': 'john@example.com'},
);
```

FLUTTER - DIO
Ejemplos de uso común

Métodos HTTP

```
// PUT
Response response = await dio.put(
  '/users/1',
  data: {'name': 'John Updated'},
);

// DELETE
Response response = await dio.delete('/users/1');
```

FLUTTER - DIO
Ejemplos de uso común

Manejo de errores

```
try {
  Response response = await dio.get('/users');
} on DioException catch (e) {
  if (e.response != null) {
    print('Error del servidor: ${e.response!.statusCode}');
    print('Mensaje: ${e.response!.data}');
  } else {
    print('Error de red: ${e.message}');
  }
}
```

FLUTTER - DIO
Ejemplos de uso común

Query parameters

```
Response response = await dio.get(
  '/users',
  queryParameters: {
    'page': 1,
    'limit': 10,
    'sort': 'name',
  },
);
```

FLUTTER - DIO
Ejemplos de uso común

Headers personalizados

```
Response response = await dio.get(
  '/profile',
  options: Options(headers: {
    'Authorization': 'Bearer your_token',
    'Custom-Header': 'value',
  }),
);
```

FLUTTER – DIO
Ejemplos de uso común

Logger Interceptor

```
dio.interceptors.add(LogInterceptor(
  request: true,
  requestHeader: true,
  requestBody: true,
  responseHeader: true,
  responseBody: true,
  error: true,
));
```

FLUTTER - DIO
Ejemplos de uso común

Auth Interceptor

```
dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) async {
    final token = await getTokenFromStorage();
    options.headers['Authorization'] = 'Bearer $token';
    return handler.next(options);
  },
  onError: (DioException e, handler) async {
    if (e.response?.statusCode == 401) {
      // Token expirado, refrescar
      await refreshToken();
      // Reintentar request
      return handler.resolve(await dio.fetch(e.requestOptions));
    }
    return handler.next(e);
  },
));
```

FLUTTER - DIO
Consideraciones de performance

DIO es más pesado que el paquete http nativo, pero ofrece muchas más funcionalidades. Es ideal para aplicaciones que necesitan:

Manejo complejo de autenticación

- **Upload/Download de archivos**
- **Manejo avanzado de errores**
- **Transformación de datos**
- **Requests concurrentes complejas**

FLUTTER - DIO

Conclusión

DIO es la elección perfecta cuando necesitas funcionalidades HTTP avanzadas en Flutter. Su sistema de interceptores, manejo de errores y características adicionales lo hacen superior al paquete http nativo para aplicaciones complejas.

G R A C I A S

Línea de atención al ciudadano: 01 8000 910270
Línea de atención al empresario: 01 8000 910682

@SENAComunica

www.sena.edu.co